



Assignment 05: Linguistic Preprocessing of a Kafka Novel

handed out: 21 May, 14:00
to be submitted by: 28 May, 12:00

In this assignment, we are going to explore some simple linguistic phenomena in the classic Austrian/Czech novel *Der Prozess* (“The Trial”) by Franz Kafka, first published posthumously in 1925. It tells the story of K., who gets prosecuted by an inscrutable judicial system without ever finding out what his crime is supposed to be. The raw text of the novel and an English translation are provided as `kafka_1925_der-prozess.txt` and `kafka_1925_the-trial.txt` (both are in the public domain, distributed by Project Gutenberg).

Task 1: Parsing and Length Comparisons

Install SpaCy and let it download the models `en_core_web_md` and `de_core_news_md` for English and German, respectively. Create a new Jupyter notebook and import `spacy`. Load the contents of the two files into a single string each, making sure to replace all newline characters in the raw text by a space (otherwise sentence segmentation will not work well). Run the relevant models on the results, and inspect the first few full sentences in both versions to check whether the outputs make sense. How many tokens are there in the English and German versions? And how does the number of sentences compare in both versions?

Task 2: Understanding Differences in the Usage of Nouns

- Familiarise yourself with the different properties of the SpaCy `Token` object. Use two instances of `collections.Counter` in order to count the occurrences of nouns in both texts (making sure to use lemmatisation!), and print out the most frequent nouns in both versions. Does the English list match your expectations? Are there any surprises (in case you have not read the novel)?
- If you know German, you will have noticed that the rankings differ more than we might perhaps expect of texts that are semantically equivalent (within the limits afforded by translation). The most striking example is that *Richter* only occurs about half as often as its English equivalent *judge*. One potential reason for this is the strong tendency of German to use compounds, which are written as single words. Use your German noun counter to find all instances of compounds involving *-richter* as the second component. What do you find? Look up the meanings of any German compound you do not understand.
- Ideally, we would be able to work with sentence-aligned versions in order to find the English translations of our compounds. In this simple case, we can get away without that by just retrieving the tokens preceding each instance of *judge* in the English translation. Build another counter of the resulting bigrams, and check the results in order to see whether they substantiate the suspicion that compounding is likely to explain most of the differences between the frequency rankings.

Task 3: Extracting Verb-Subject and Verb-Object Pairs

For the next task, you will need to extract all pairs of verbs and the heads of their main arguments (subject and object) by means of the dependency annotations in the parsed English version (working with the German version will be difficult if you can’t judge the outputs).

- Make yourself familiar with how dependency annotations are encoded, and use them to extract lemmatised pairs of full verbs (POS tag `VERB`) and their subjects (relation `nsubj`), storing them as tuples for later processing. The first pair in my solution is `('tell', 'trial')` (actually due to a misparse of the title), the second more reasonable pair is `('tell', 'someone')`.
- Repeat the process for verb-object pairs (relation `dobj`). The first pair should be `('tell', 'lie')`, reflecting the sentence *Somebody must have been telling lies about Josef K. [...]*.

Task 4: A Survey of Activities Performed

Use the verb-subject and verb-object pairs you extracted in Task 3 in order to answer the following simple questions about the activities described in the novel (if you use list comprehension in order to generate the argument to a `Counter` constructor, each of these can be one-liners):

- What are the five most frequent transitive verbs?
- What are the ten most common subjects of *say*?
- Which ten actions does the main protagonist ("K.") perform the most?
- Which ten actions does the main protagonist ("K.") undergo the most?
- What do lawyers in the novel appear to do most of the time?
- What happens to lawyers in the novel?

Interpret the results of each query in the light of what you know about the contents of the novel.

Why might the analysis as we performed it miss many activities in which the main protagonist or the lawyers participate? Which additional type of linguistic preprocessing (currently not supported by the models) would help resolve this issue?

Task 5: Comparing Usage of the Passive Voice

Coming back to a linguistic question, we will analyse the tendencies of verbs in both languages to appear either in active or in passive voice. The problem is that as the passive is a periphrastic construction in both languages, there is no morphological marking which would help retrieval. Instead, we need to detect constructions consisting of certain auxiliaries and participles. A further complicating factor is that the dependency formalisms used by the two models are very different, including the basic decision whether the auxiliary or the main verb is the head of a complex verb form.

- Go through all occurrences of verbs (POS tag `VERB`) in the English version, and determine whether they are active or passive forms. In order to do this, you have to go through the dependents (`children`) and look for auxiliaries (POS tag `AUX`), storing the lemmas of any auxiliaries you encounter. If there is an auxiliary which is a form of the lemma *be*, and the main verb is a past participle (`Tense` is `Past`), you can classify this verb as being in passive voice. All other verbs can be assumed to be in active voice. Store the results as a list of quadruples of the format `("eng", token.idx, <lemma>, "active" or "passive")`. First entries of both categories are `('eng', 210, 'tell', 'active')` and `('eng', 298, 'arrest', 'passive')`.
- To repeat the same extraction for German, we need to match a very similar pattern, but due to the different dependency formalism, the logic has to be a bit different. This time, we check for each full verb (POS tag `VERB`) whether it depends on an auxiliary. Retrieve the head of the verb using `token.head`, and retrieve the lemma in case it is an auxiliary (POS tag `AUX`). If the lemma of the auxiliary is *werden*, we again check whether the main verb is a past participle (`Tense` is `Past`). If both conditions are met, we have found a passive construction, otherwise the verb will be in active voice. As before, store the results as a list of quadruples of the format `("deu", token.idx, <lemma>, "active" or "passive")`. In my solution, the first entries for both categories are `('deu', 327, 'verleumden', 'active')` (*verleumden* is translated to *tell lies about* in our version) and `('deu', 410, 'verhaften', 'passive')` (*verhaften* means “to arrest”).
- Load the two collections of tuples into a Pandas dataframe with column names `Language`, `Position`, `Lemma`, `Voice`. Compute the overall distribution of voice usage in each language, and compare the two figures. Is there a marked difference, and if so, do you have any idea what might be the reason?
- For each lemma, use Pandas features to compute the percentage of occurrences in passive voice. Rank the lemmas in each language in descending order by this “passive ratio”, and visualise the ten “most passive verbs” in separate graphs for every language. Ideally, this should be done in horizontal bar plots. Look up the meanings of the German verbs - is there any marked difference between the semantics of the verbs which are passivised the most?